Découverte et pratique guidée de React : Composants, JSX et architecture d'une application avec CRA

Partie 1 — ES6

Plan guidé

1. **Introduction rapide à ES6**
   - Pourquoi ES6 ? React utilise beaucoup de let/const, fonctions fléchées, template literals, destructuring, modules.
   - Lien W3Schools : https://www.w3schools.com/Js/js_es6.asp
2. **Exercice pratique en direct**

   Variables :

   ```
   let prenom = "Ali";
   const age = 25;
   console.log(prenom, age);
   ```

   - **Fonctions fléchées :**

   ```
   const addition = (a, b) => a + b;
   console.log(addition(3, 4));
   ```

   Template literals :

   ```
   console.log(`Bonjour ${prenom}, tu as ${age} ans.`);
   ```

Destructuring :

```
const personne = { nom: "Sara", age: 30 };
const { nom, age } = personne;
console.log(nom, age);
```

Modules (import/export) → très rapide, juste montrer le concept, pas d'exercice complexe.

Partie 2 — Découverte pratique de React

Étape 1 — Ouvrir le projet CRA

1. Ouvrir VS Code → Terminal intégré.
2. Se placer dans le projet :

```
cd mon-projet
npm start
```

3. Vérifier navigateur → page React par défaut.
   Voici notre point de départ. App.js est le composant racine.

Étape 2 — Explorer l'architecture

1. Ouvrir chaque dossier/fichier dans VS Code :
   - public/index.html → fichier HTML statique
   - src/index.js → point d'entrée React

- src/App.js → composant racine
- src/components/ → dossier pour futurs composants

CRA crée tout pour nous, on va explorer chaque élément avant d'écrire du code.

Étape 3 — Créer le premier composant

1. Créer Bienvenue.js dans src/components/ :

```
function Bienvenue() {
  return <h1>Bienvenue dans React !</h1>;
}
export default Bienvenue;
```

2. Importer dans App.js et afficher :

```
import Bienvenue from "./components/Bienvenue";

function App() {
  return (
    <div>
      <Bienvenue />
    </div>
  );
```

```
}
export default App;
```

3. Vérifier dans le navigateur → texte visible.

*Les composants sont réutilisables et commencent toujours par une **majuscule**.*

Étape 4 — Explorer JSX

1. Ajouter des éléments dans **App.js** :

```
function App() {
  const prenom = "Ali";
  return (
    <div>
      <Bienvenue />
      <p>Bonjour {prenom} !</p>
      <div>
        <h2>Imbrication JSX</h2>
        <p>Paragraphe imbriqué</p>
      </div>
    </div>
  );
```

}

2. Vérifier navigateur → tout doit s'afficher.

JSX mélange HTML et JS. Le Virtual DOM met à jour seulement ce qui change.

Étape 5 — Explorer App.js et index.js

1. Ouvrir src/index.js :

```
import ReactDOM from "react-dom/client";
import App from "./App";

const root =
ReactDOM.createRoot(document.getElementById("root"));
root.render(<App />);
```

2. Expliquer :
   - root.render(<App />) → insertion de App dans DOM réel via Virtual DOM
   - React ne touche jamais directement le DOM complet → performant

Étape 6 — Imbrication et mini-modification

1. Modifier Bienvenue.js pour ajouter un paragraphe :

```
function Bienvenue() {
  return (
    <div>
      <h1>Bienvenue dans React !</h1>
      <p>Vous êtes prêts à coder avec JSX et des composants.</p>
    </div>
  );
}
```

2. Vérifier navigateur → texte mis à jour instantanément.

React met à jour seulement ce qui change grâce au Virtual DOM.

Étape 7 — Ajouter plusieurs composants simples

Comprendre la réutilisation des composants.

1. Créer un nouveau composant Message.js dans src/components/ :

```
function Message() {
  return <p>Ceci est mon premier message React !</p>;
}


export default Message;
```

2. Dans App.js, importer et utiliser ce composant plusieurs fois :

```
import Bienvenue from "./components/Bienvenue";
import Message from "./components/Message";

function App() {
  return (
    <div>
      <Bienvenue />
      <Message />
      <Message />
      <Message />
    </div>
  );
}
```

3. **Vérifier dans le navigateur** → trois messages identiques s'affichent.

On peut réutiliser un composant autant de fois qu'on veut, sans réécrire le code.

Étape 8 — Modifier le contenu avec variables

Voir comment JSX peut mélanger JS et HTML.

1. Modifier Message.js :

```
function Message() {
  const texte = "Ceci est un message dynamique !";
  return <p>{texte}</p>;
}

export default Message;
```

2. Vérifier → le texte s'affiche depuis la variable.

Dans JSX, on peut afficher des variables JS avec { }."

Étape 9 — Imbrication de composants

Comprendre que les composants peuvent contenir d'autres composants.

1. Modifier Bienvenue.js pour inclure Message :

```
import Message from "./Message";

function Bienvenue() {
```

```
  return (
    <div>
      <h1>Bienvenue dans React !</h1>
      <p>Vous êtes prêts à coder avec JSX et des composants.</p>
      <Message />
    </div>
  );
}

export default Bienvenue;
```

2. Vérifier → le message apparaît **à l'intérieur** du composant Bienvenue.

Les composants peuvent s'imbriquer. React gère le Virtual DOM pour tout mettre à jour efficacement.

Étape 10 — Expressions simples dans JSX

Se familiariser avec les calculs et conditions simples.

1. Dans App.js :

```
function App() {
  const a = 5;
```

```
  const b = 3;
  return (
    <div>
      <Bienvenue />
      <p>5 + 3 = {a + b}</p>
      <p>{a > b ? "5 est plus grand que 3" : "3 est plus grand que 5"}</p>
    </div>
  );
}
```

2. Vérifier → les calculs et conditions s'affichent correctement.

JSX peut contenir des expressions JavaScript, même des conditions ternaires.

Étape 11 — Ajouter des éléments HTML variés

S'habituer aux différents éléments JSX.

1. Dans App.js, ajouter :

```
function App() {
  return (
    <div>
```

```
    <Bienvenue />
    <ul>
      <li>Élément 1</li>
      <li>Élément 2</li>
      <li>Élément 3</li>
    </ul>
    <button>Cliquer moi</button>
  </div>
 );
}
```

2. Vérifier → la liste et le bouton apparaissent.

JSX accepte tous les éléments HTML classiques et on peut les imbriquer comme on veut.

Étape 12 — Jouer avec le style

Comprendre comment ajouter du CSS directement dans JSX.

1. Dans App.js :

```
function App() {
  const styleTitre = { color: "blue", fontSize: "24px" };
  return (
```

```
    <div>
      <h1 style={styleTitre}>Titre stylé en React</h1>
      <Bienvenue />
    </div>
  );
}
```

2. Vérifier → le titre apparaît en bleu et plus grand.

Le style peut être ajouté directement en JSX avec des objets JavaScript.